一种结合LVS的TX_XDP技术的加速与回退四层负载均衡优化策略

中国联通软件研究院
温怀湘 石鹏飞

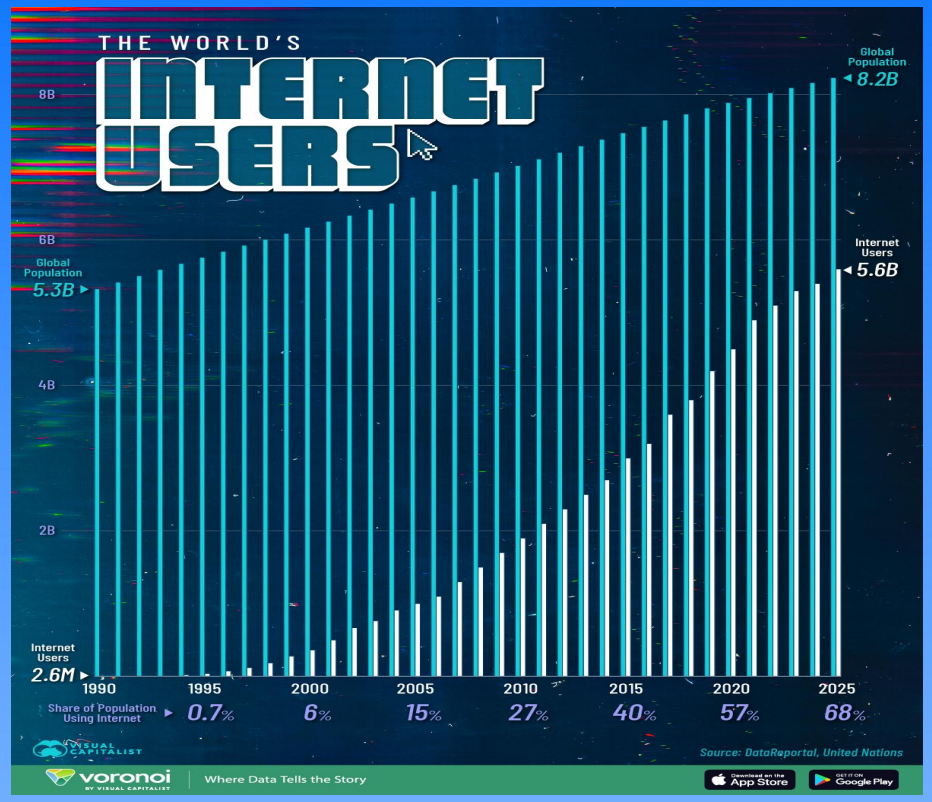
The 4th eBPF Developer Conference (eBPFDC 2026)

陕西 · 西安　　2026年4月18日

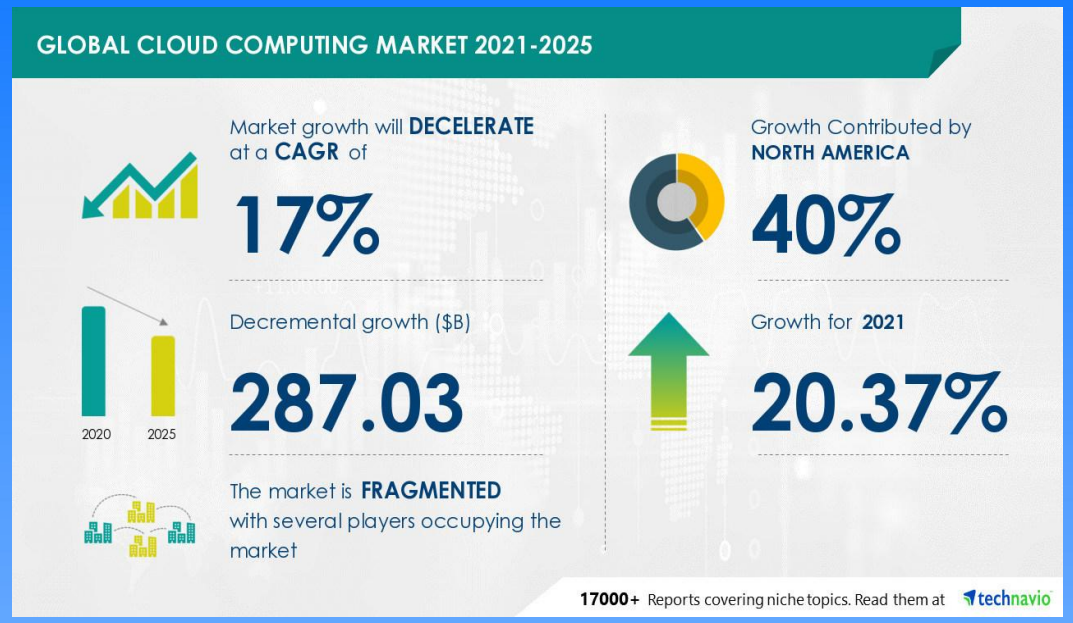
https://www.ebpftravel.com

1 Background
2 Solution Design
3 Summary
4 Outlook

Rapid growth of Internet User and Cloud Computing

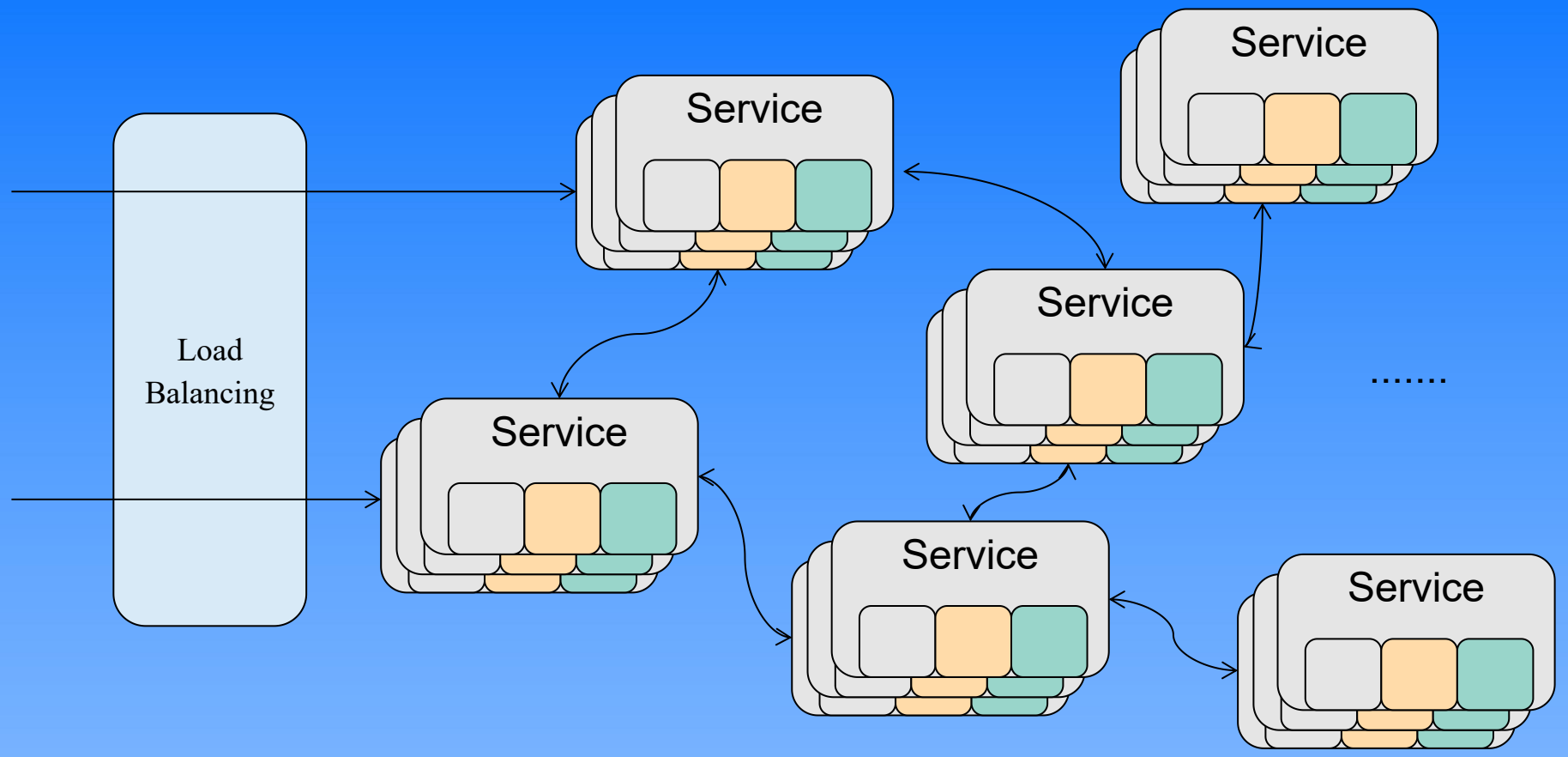


Visualized: The Growth of Global Internet Users (1990–2025)



Cloud Computing Market by Service and Geography - Forecast and Analysis 2021-2025

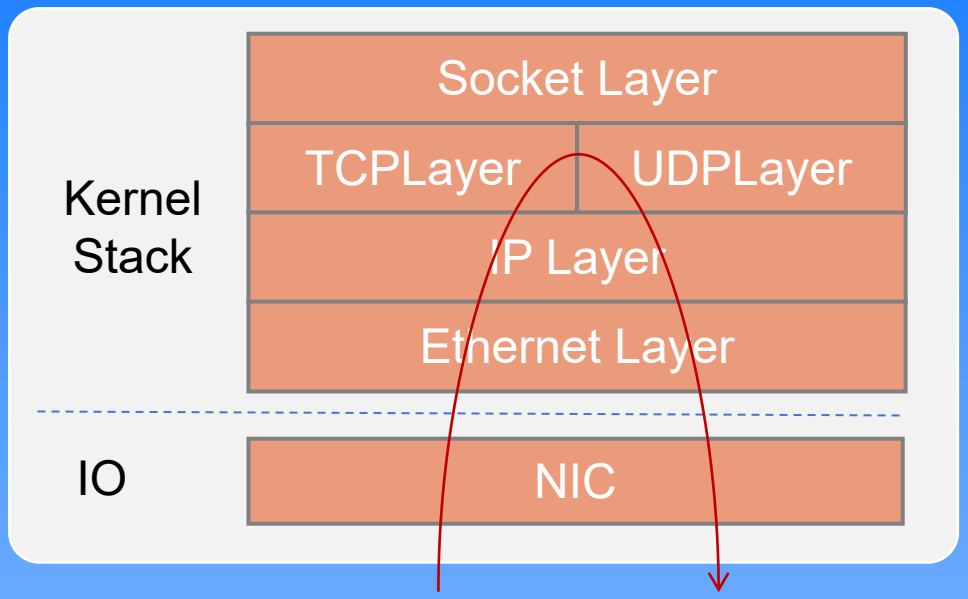
Demands on high-performance and low-latency load balancing



A Microservices-based Solution with LoadBalance Architectur

Load Balancing Solution

LVS



Full kernel networking stack

Memory Copy

High levels of concurrency result in **increased latency**

Available Optimization Techniques

DPDK

RDMA

TC

XDP

Load Balancing Solution

DPDK RDMA and AF_XDP

Maintainability： Requires re-implementing LVS load balancing logic in user space

Reliability： Lack safe fallback mechanisms

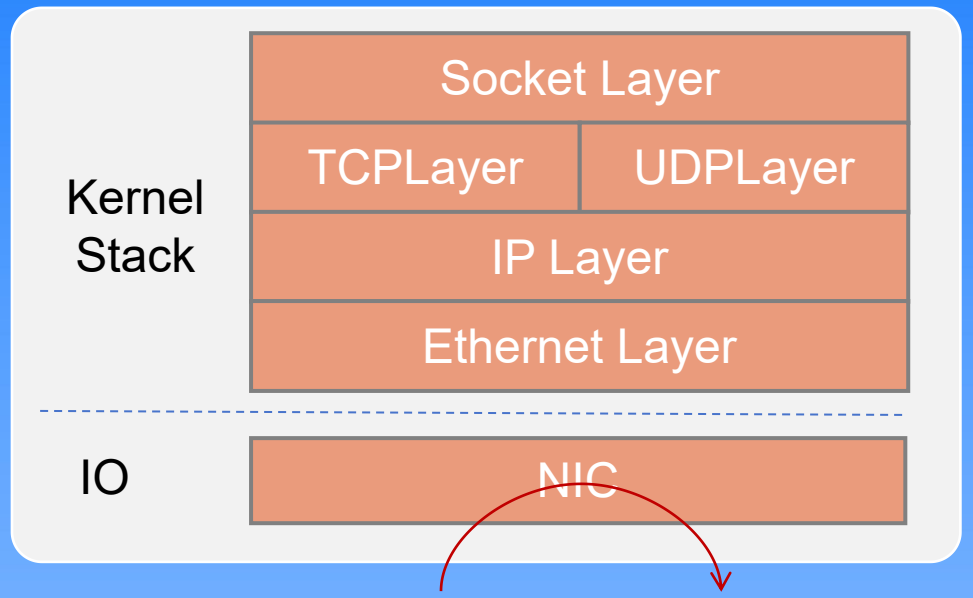
Depend on SmartNIC hardware capabilities

TC

Performance： The TC layer resides between L2 and L3, and delivers lower performance than TX

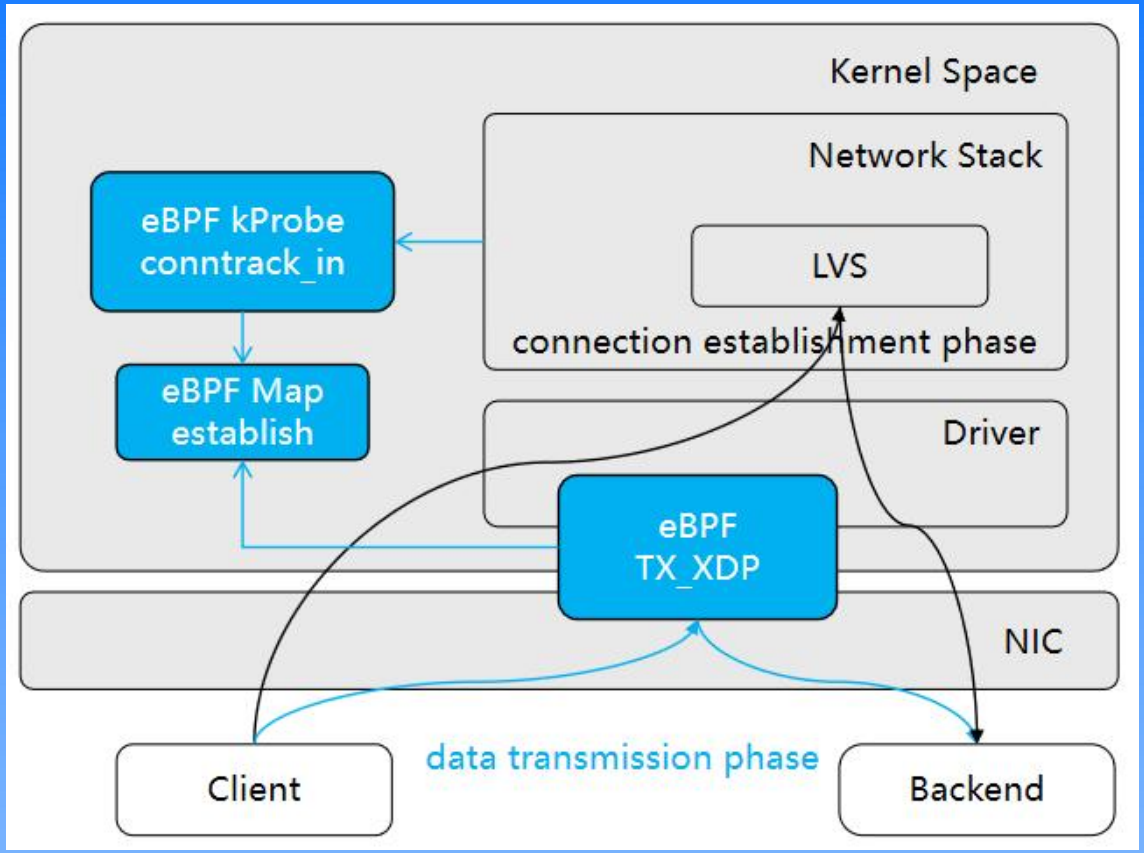
Derive

TX_XDP



1 Background
2 **Solution Design**
3 Summary
4 Outlook

Core Design



1. Real-time connection capture: A kProbe monitors the conntrack subsystem and records metadata for newly established connections.

2. Connection information sharing: Captured connection data is stored in a custom eBPF map for fast lookup.

3. High-speed packet processing: An XDP program reads the map data at the NIC layer to rewrite source/destination addresses and update checksums.

4. Kernel-bypass forwarding: The modified packet is transmitted directly via TX_XDP, enabling fast path forwarding and acceleration.

5. LVS compatibility: The entire workflow remains transparent to LVS control logic while improving data-plane performance.

Conntrack Hook Implementation

Extract Connection Information

↓

Validate Connection

↓

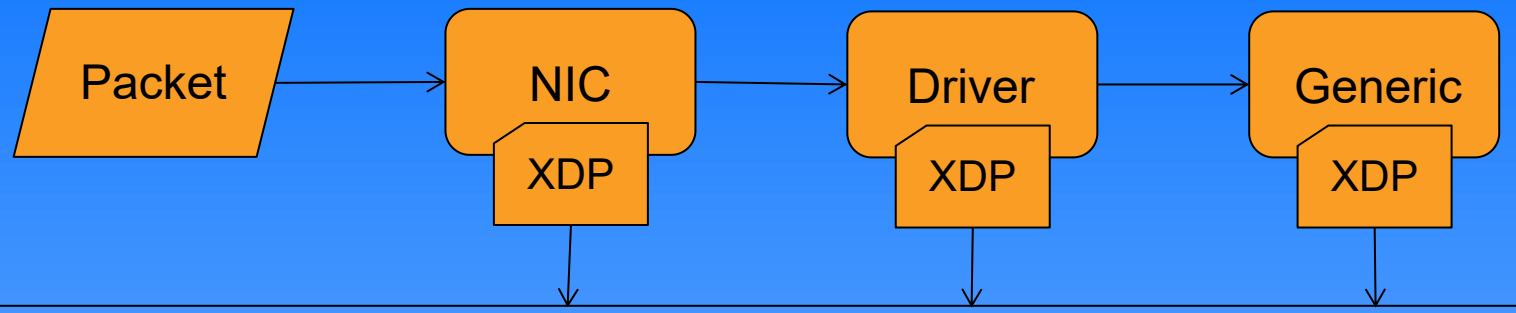
Extract and Validate Original/Reply Tuples

↓

Construct and Update conntrack_map

```
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, struct ip_tuple_t);
    __type(value, struct ip_tuple_t);
    __uint(max_entries, MAX_ENTRIES);
} conntrack_map SEC(".maps");
```

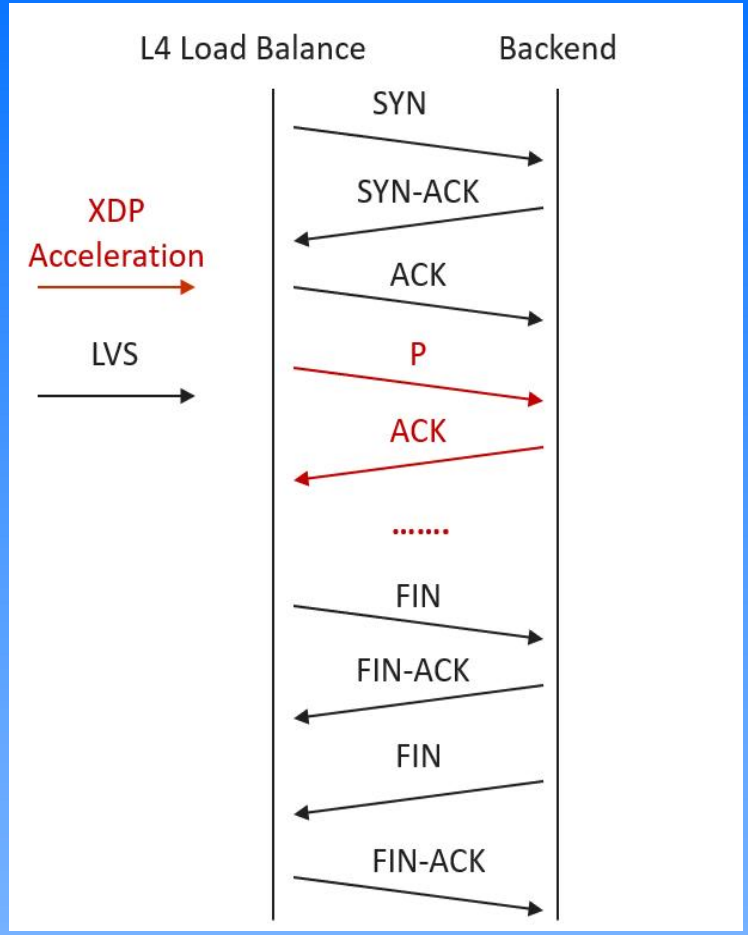
XDP Program Implementation on the NIC



1. Packet Parsing and Filtering
2. TCP RST/FIN Handling
3. SYN/SYN-ACK Processing and Context Recording
4. Packet Rewriting and Checksum Update
5. High-Speed Forwarding

```
struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ip_tuple_t);
    __type(value, struct mac_info_t);
    __uint(max_entries, MAX_ENTRIES);
} context_map SEC(".maps");
```

Packet Acceleration Diagram



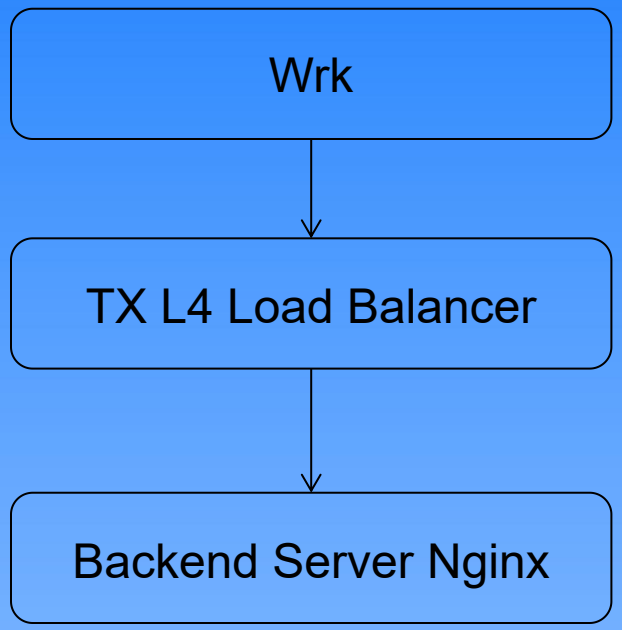
1 Background
2 Solution Design
3 **Summary**
4 Outlook

Experimental Environment

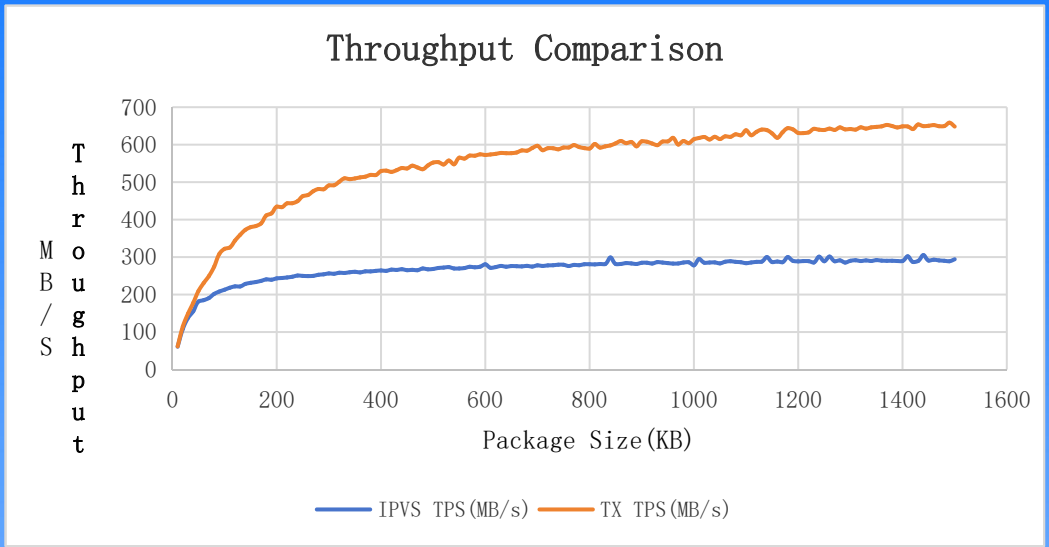
Real physical environmen

Host Model: Huawei 2288H V5
Memory: 256 GB
CPU: Intel(R) Xeon(R) Silver 4114
Network Interface Card: ConnectX-4
Operating System: UOS 1050a
Kernel Version: 5.10

Logical View



Experimental Results

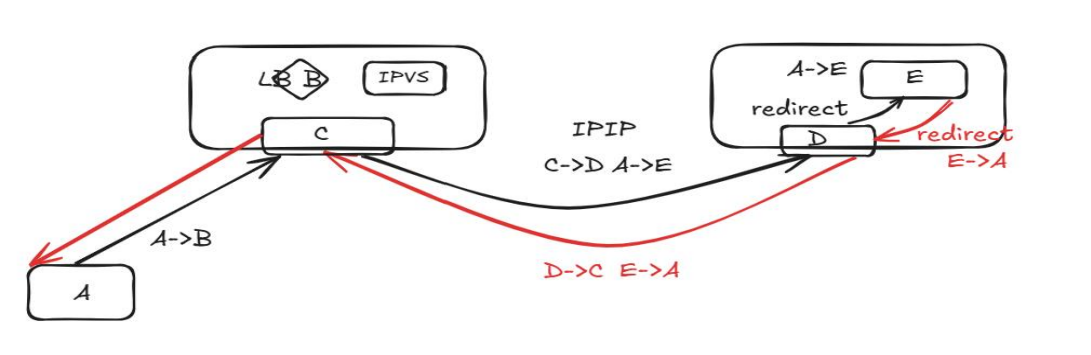


Based on the experimental findings, the proposed LVS-TX_XDP hybrid acceleration and fallback strategy demonstrates substantial performance gains in Layer-4 load balancing scenarios.

1 Background
2 Solution Design
3 Summary
4 **Outlook**

Issues to Be Optimized

As a result, when the data size per transmission is small (≤10 KB), the acceleration effect is limited and, in some cases, may even perform slightly below standard LVS.



Optimization Directions

Future work will focus on extending the acceleration coverage to the full TCP connection lifecycle, including the connection establishment and termination phases, thereby further optimizing end-to-end performance.

CCIOT '25: Proceedings of the 2025 10th International Conference on Cloud Computing and Internet of Things
Pages 92 - 98 https://doi.org/10.1145/3785520.3785533

Thank you